



MEDIACREST
TRAINING COLLEGE



Statistical Foundation for Data Analysis and Machine Learning

Introduction to Linear
Regression Analysis



FOUNDATIONS
FOR DATA-DRIVEN
DECISIONS



STATISTICAL INSIGHT
REAL-WORLD
APPLICATION



BUILDING BLOCKS FOR
MACHINE LEARNING
EXCELLENCE

STATISTICAL DATA ANALYSIS MACHINE LEARNING Comprehensive Training Notebook

In [60]:

```
#!pip install plotnine
```

In [24]:

```
# **Cell 1: Import Libraries**
import pandas as pd
import numpy as np
import statsmodels.api as sm
import warnings

warnings.filterwarnings("ignore")

from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan, acorr_breusch_godfrey
from statsmodels.stats.stattools import jarque_bera, durbin_watson
```

In [61]:

```
from plotnine.data import mtcars
# Create a copy
mtcars = mtcars.copy()
# Display the first few rows
mtcars.head()
```

Out[61]:

	name	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

In [62]:

```
# Import libraries
from plotnine.data import mtcars
from scipy.stats import pearsonr, spearmanr

# Load dataset
df = mtcars.copy()

# Select variables
x = df['wt']
y = df['mpg']

# Pearson correlation
pearson_corr, pearson_p = pearsonr(x, y)

# Spearman correlation
spearman_corr, spearman_p = spearmanr(x, y)

# Display results
print("="*50)
print("Pearson Correlation")
print("="*50)
print(f"Correlation coefficient (r): {pearson_corr:.4f}")
print(f"P-value: {pearson_p:.6f}")

print("\n" + "="*50)
```

```
print("Spearman Correlation")
print("="*50)
print(f"Correlation coefficient (ρ): {spearman_corr:.4f}")
print(f"P-value: {spearman_p:.6f}")
```

```
=====
Pearson Correlation
=====
```

```
Correlation coefficient (r): -0.8677
P-value: 0.000000
```

```
=====
Spearman Correlation
=====
```

```
Correlation coefficient (ρ): -0.8864
P-value: 0.000000
```

In [63]:

```
df = mtcars.copy()

# Pearson correlation
pearson_corr = df['wt'].corr(df['mpg'], method='pearson')

# Spearman correlation
spearman_corr = df['wt'].corr(df['mpg'], method='spearman')

print(f"Pearson correlation : {pearson_corr:.4f}")
print(f"Spearman correlation: {spearman_corr:.4f}")
```

```
Pearson correlation : -0.8677
Spearman correlation: -0.8864
```

In [64]:

```
# Import libraries
# Load dataset
df = mtcars.copy()

# Select variables
corr_data = df[['mpg', 'cyl', 'disp', 'hp', 'wt']]

# Pearson correlation matrix
corr_matrix = corr_data.corr(method='pearson')

# Display correlation matrix
print(corr_matrix.round(3))
```

	mpg	cyl	disp	hp	wt
mpg	1.000	-0.852	-0.848	-0.776	-0.868
cyl	-0.852	1.000	0.902	0.832	0.782
disp	-0.848	0.902	1.000	0.791	0.888
hp	-0.776	0.832	0.791	1.000	0.659
wt	-0.868	0.782	0.888	0.659	1.000

In [65]:

```
# Display correlation matrix with 3 decimal places
corr_matrix.style.format("{:.3f}")
```

Out[65]:

	mpg	cyl	disp	hp	wt
mpg	1.000	-0.852	-0.848	-0.776	-0.868

	mpg	cyl	disp	hp	wt
cyl	-0.852	1.000	0.902	0.832	0.782
disp	-0.848	0.902	1.000	0.791	0.888
hp	-0.776	0.832	0.791	1.000	0.659
wt	-0.868	0.782	0.888	0.659	1.000

In [67]:

```
import matplotlib.pyplot as plt

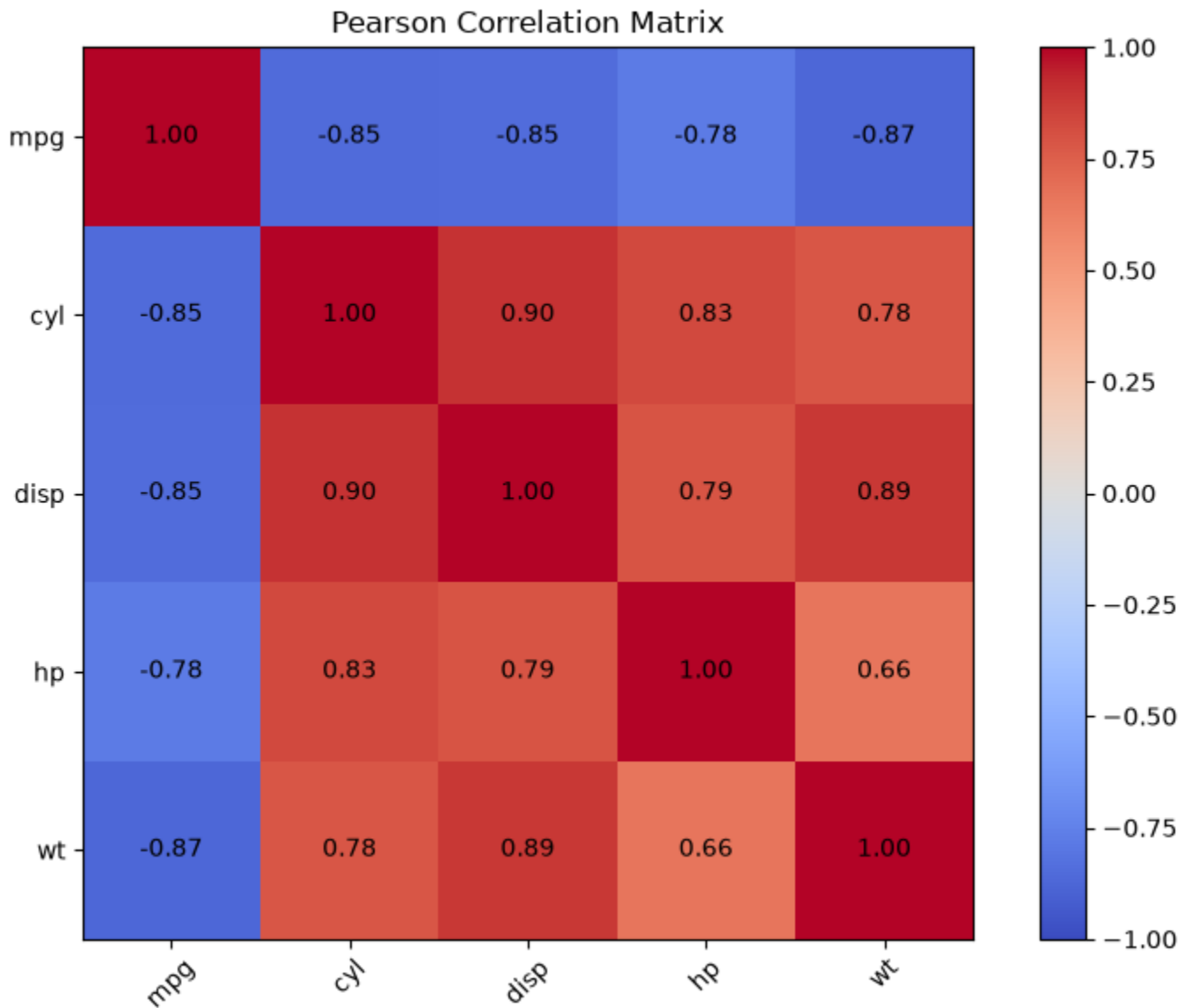
corr = corr_data.corr()

plt.figure(figsize=(8, 6))
plt.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)

plt.xticks(range(len(corr)), corr.columns, rotation=45)
plt.yticks(range(len(corr)), corr.columns)

for i in range(len(corr)):
    for j in range(len(corr)):
        plt.text(j, i, f"{corr.iloc[i, j]:.2f}",
                 ha='center', va='center')

plt.colorbar()
plt.title("Pearson Correlation Matrix")
plt.tight_layout()
plt.show()
```



LINEAR REGRESSION

Simple Linear Regression Using mtcars Data

In [68]:

```
# Import libraries
from plotnine.data import mtcars
import statsmodels.api as sm

# Load dataset
df = mtcars.copy()

# Define predictor and response
X = df['wt']
y = df['mpg']

# Add intercept (constant)
X = sm.add_constant(X)

# Fit Simple Linear Regression model
model = sm.OLS(y, X).fit()
```

```
# Display regression results
print(model.summary())
```

OLS Regression Results						
Dep. Variable:	mpg	R-squared:	0.753			
Model:	OLS	Adj. R-squared:	0.745			
Method:	Least Squares	F-statistic:	91.38			
Date:	Wed, 01 Jul 2026	Prob (F-statistic):	1.29e-10			
Time:	19:56:24	Log-Likelihood:	-80.015			
No. Observations:	32	AIC:	164.0			
Df Residuals:	30	BIC:	167.0			
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	37.2851	1.878	19.858	0.000	33.450	41.120
wt	-5.3445	0.559	-9.559	0.000	-6.486	-4.203
Omnibus:	2.988	Durbin-Watson:	1.252			
Prob(Omnibus):	0.225	Jarque-Bera (JB):	2.399			
Skew:	0.668	Prob(JB):	0.301			
Kurtosis:	2.877	Cond. No.	12.7			

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Simple Linear Regression with Hypertension Data

```
In [69]:
df = pd.read_csv("htn_dat.csv")
df.head()
```

	ID	DBP	SBP	BMI	age	married	male.gender	hgb_centered	adv_HIV	survtime	e
0	16097	60	90	NaN	28.000000	0.0	0	NaN	NaN	338	
1	930	75	110	27.339912	26.458590	1.0	0	-3.9	0.0	439	
2	6877	60	80	17.689789	42.951405	0.0	0	-3.2	NaN	752	
3	36464	60	90	19.854080	50.149212	1.0	1	NaN	NaN	526	
4	36180	60	100	21.295294	30.554415	0.0	1	-0.4	NaN	215	

```
In [70]:
print(df.isnull().sum())

ID          0
DBP         0
SBP         0
BMI        670
age         0
married     164
male.gender  0
```

```

hgb_centered      1394
adv_HIV           1961
survtime          0
event             0
arv_naive         0
urban.clinic      0
log_creat_centered 1538
IPW_weight        0
SBP_ge120         0
dtype: int64

```

In []:

```
df = df.dropna()
```

In [71]:

```
df[df.isnull().any(axis=1)]
```

Out[71]:

	ID	DBP	SBP		BMI	age	married	male.gender	hgb_centered	adv_HIV	survtime
0	16097	60	90		NaN	28.000000	0.0	0	NaN	NaN	338
2	6877	60	80	17.689789	42.951405		0.0	0	-3.2	NaN	752
3	36464	60	90	19.854080	50.149212		1.0	1	NaN	NaN	526
4	36180	60	100	21.295294	30.554415		0.0	1	-0.4	NaN	215
5	7923	60	120	16.880194	20.372347		1.0	0	-4.0	NaN	272
...	...	...	...	...	...	...	...	...	...	...	...
4988	19660	60	120		NaN	47.329227	0.0	0	-0.4	NaN	357
4992	25397	70	130		NaN	21.965776	1.0	0	NaN	NaN	1828
4994	33428	75	110	16.233072	41.639973		1.0	0	NaN	1.0	646
4996	1605	78	120	21.936348	39.912388		1.0	0	-1.7	NaN	1
4997	39234	80	110	18.591131	24.194387		1.0	1	3.5	1.0	414

2673 rows × 16 columns

In [72]:

```
df = df.dropna()
```

In [73]:

```
print(df.isnull().sum())
```

```

ID              0
DBP             0
SBP             0
BMI             0
age            0
married        0
male.gender    0
hgb_centered   0
adv_HIV        0
survtime       0
event          0
arv_naive      0

```

```
urban.clinic      0
log_creat_centered 0
IPW_weight        0
SBP_ge120         0
dtype: int64
```

In [74]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
Index: 2326 entries, 1 to 4998
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                    2326 non-null   int64
1   DBP                  2326 non-null   int64
2   SBP                  2326 non-null   int64
3   BMI                  2326 non-null   float64
4   age                  2326 non-null   float64
5   married              2326 non-null   float64
6   male.gender          2326 non-null   int64
7   hgb_centered         2326 non-null   float64
8   adv_HIV              2326 non-null   float64
9   survtime             2326 non-null   int64
10  event                2326 non-null   int64
11  arv_naive            2326 non-null   int64
12  urban.clinic         2326 non-null   int64
13  log_creat_centered   2326 non-null   float64
14  IPW_weight           2326 non-null   float64
15  SBP_ge120            2326 non-null   int64
dtypes: float64(7), int64(9)
memory usage: 308.9 KB
```

In [75]:

```
X = df['BMI']
X = sm.add_constant(X)

y = df['SBP']
```

In [76]:

```
### Cell 4: Fit the Model
modell = sm.OLS(y, X).fit()
print(modell.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          SBP      R-squared:                0.041
Model:                  OLS      Adj. R-squared:           0.041
Method:                 Least Squares      F-statistic:           100.2
Date:                  Wed, 01 Jul 2026      Prob (F-statistic):       3.96e-23
Time:                  20:06:20      Log-Likelihood:          -9623.3
No. Observations:      2326      AIC:                    1.925e+04
Df Residuals:          2324      BIC:                    1.926e+04
Df Model:               1
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	92.8415	1.872	49.587	0.000	89.170	96.513
BMI	0.8572	0.086	10.012	0.000	0.689	1.025


```
=====
Omnibus:                641.236    Durbin-Watson:                2.060
Prob(Omnibus):           0.000    Jarque-Bera (JB):            2483.156
Skew:                   1.306    Prob(JB):                     0.00
Kurtosis:               7.336    Cond. No.                    131.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Multiple Linear Regression-mtcars

In [77]:

```
df = mtcars.copy()
X = df[['wt', 'hp']]
X = sm.add_constant(X)

y = df['mpg']
```

In [78]:

```
modell = sm.OLS(y, X).fit()
print(modell.summary())
```

OLS Regression Results

```
=====
Dep. Variable:            mpg    R-squared:                0.827
Model:                    OLS    Adj. R-squared:           0.815
Method:                    Least Squares    F-statistic:             69.21
Date:                      Wed, 01 Jul 2026    Prob (F-statistic):       9.11e-12
Time:                      20:10:24    Log-Likelihood:          -74.326
No. Observations:          32    AIC:                     154.7
Df Residuals:              29    BIC:                     159.0
Df Model:                  2
Covariance Type:            nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	37.2273	1.599	23.285	0.000	33.957	40.497
wt	-3.8778	0.633	-6.129	0.000	-5.172	-2.584
hp	-0.0318	0.009	-3.519	0.001	-0.050	-0.013

```
=====
Omnibus:                5.303    Durbin-Watson:                1.362
Prob(Omnibus):           0.071    Jarque-Bera (JB):            4.046
Skew:                   0.855    Prob(JB):                     0.132
Kurtosis:               3.332    Cond. No.                    588.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [79]:

```
### VIF
vif = pd.DataFrame()

vif["Variable"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i)
```

```
for i in range(X.shape[1]):  
  
print(vif)
```

	Variable	VIF
0	const	12.161539
1	wt	1.766625
2	hp	1.766625

In []:

In [80]:

```
# **Cell 2: Load the Data**  
df = pd.read_csv("income.csv")  
df1 = pd.read_csv("training model.csv")  
df2 = pd.read_csv("demand_data.csv")
```

In [81]:

```
df.head()
```

Out[81]:

	Year	Income	Consumption
0	1950.0	791.8	733.2
1	1951.0	819.0	748.7
2	1952.0	844.3	771.4
3	1953.0	880.0	802.5
4	1954.0	894.0	822.7

In [82]:

```
print(df.isnull().sum())
```

```
Year      1  
Income    1  
Consumption 1  
dtype: int64
```

In [83]:

```
df[df.isnull().any(axis=1)]
```

Out[83]:

	Year	Income	Consumption
36	NaN	NaN	NaN

In [84]:

```
df = df.dropna()
```

In [85]:

```
# MODEL  
## Simple Linear Regression  
### Cell 3: Prepare the Variables  
X = df[["Income"]]  
X = sm.add_constant(X)
```

```
y = df["Consumption"]
```

In [86]:

```
### Cell 4: Fit the Model
modell = sm.OLS(y, X).fit()
print(modell.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          Consumption    R-squared:                0.999
Model:                  OLS           Adj. R-squared:           0.999
Method:                 Least Squares  F-statistic:              2.359e+04
Date:                   Wed, 01 Jul 2026  Prob (F-statistic):      6.61e-50
Time:                   21:52:58        Log-Likelihood:           -155.71
No. Observations:       36             AIC:                     315.4
Df Residuals:           34             BIC:                     318.6
Df Model:               1
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	11.3737	9.629	1.181	0.246	-8.196	30.943
Income	0.8983	0.006	153.603	0.000	0.886	0.910

```
=====
Omnibus:                 4.260    Durbin-Watson:           0.843
Prob(Omnibus):            0.119    Jarque-Bera (JB):       3.195
Skew:                    -0.383    Prob(JB):               0.202
Kurtosis:                 4.242    Cond. No.               5.05e+03
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 5.05e+03. This might indicate that there are strong multicollinearity or other numerical problems.

In [87]:

```
### Cell 5: Heteroscedasticity Test
bp_test = het_breuschpagan(modell.resid, modell.model.exog)

print("LM Statistic :", bp_test[0])
print("LM p-value   :", bp_test[1])
print("F Statistic  :", bp_test[2])
print("F p-value    :", bp_test[3])

LM Statistic : 6.805057079710659
LM p-value   : 0.009090004781862582
F Statistic  : 7.925069123850486
F p-value    : 0.008053028131628304
```

In [88]:

```
### Cell 6: Normality Test
jb_test = jarque_bera(modell.resid)

print("JB Statistic :", jb_test[0])
print("p-value      :", jb_test[1])
print("Skewness     :", jb_test[2])
print("Kurtosis      :", jb_test[3])
```

JB Statistic : 3.1949075319392355
p-value : 0.20241124881385675
Skewness : -0.38315900493444455
Kurtosis : 4.242052761510855

In [89]:

```
### Cell 7: Durbin-Watson Test
dw = durbin_watson(model1.resid)
print("Durbin-Watson Statistic =", dw)
```

Durbin-Watson Statistic = 0.8430634591987601

In [90]:

```
### Cell 8: Breusch-Godfrey Test
bg = acorr_breusch_godfrey(model1, nlags=1)

print("LM Statistic :", bg[0])
print("LM p-value   :", bg[1])
print("F Statistic  :", bg[2])
print("F p-value    :", bg[3])
```

LM Statistic : 10.468043862555517
LM p-value : 0.0012145725820036156
F Statistic : 13.529924836339157
F p-value : 0.000830033530193847

In [91]:

```
# MODEL 2
## Multiple Linear Regression

### Cell 9: Prepare the Variables
X = df1[["ExchRate", "LendIntRates"]]
X = sm.add_constant(X)

y = df1["CPI"]
```

In [92]:

```
### Cell 10: Fit the Model
model2 = sm.OLS(y, X).fit()

print(model2.summary())
```

OLS Regression Results

Dep. Variable:	CPI	R-squared:	0.772			
Model:	OLS	Adj. R-squared:	0.752			
Method:	Least Squares	F-statistic:	38.96			
Date:	Wed, 01 Jul 2026	Prob (F-statistic):	4.11e-08			
Time:	21:53:56	Log-Likelihood:	-110.21			
No. Observations:	26	AIC:	226.4			
Df Residuals:	23	BIC:	230.2			
Df Model:	2					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	7.8998	15.023	0.526	0.604	-23.178	38.977
ExchRate	1.2664	0.158	8.011	0.000	0.939	1.593
LendIntRates	-1.4861	0.510	-2.912	0.008	-2.542	-0.430
=====						

Omnibus:	2.642	Durbin-Watson:	0.282
Prob(Omnibus):	0.267	Jarque-Bera (JB):	1.324
Skew:	0.492	Prob(JB):	0.516
Kurtosis:	3.502	Cond. No.	285.

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [93]:

```
### Cell 11: VIF
vif = pd.DataFrame()

vif["Variable"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i)
              for i in range(X.shape[1])]

print(vif)
```

	Variable	VIF
0	const	18.440192
1	ExchRate	1.009553
2	LendIntRates	1.009553

In [94]:

```
### Cell 12: Breusch-Pagan Test
bp_test = het_breuschpagan(model2.resid, model2.model.exog)

print("LM Statistic :", bp_test[0])
print("LM p-value   :", bp_test[1])
print("F Statistic  :", bp_test[2])
print("F p-value    :", bp_test[3])
```

```
LM Statistic : 8.237265392765991
LM p-value   : 0.01626674081694156
F Statistic  : 5.3329937147307245
F p-value    : 0.012507471142798897
```

In [95]:

```
### Cell 13: Jarque-Bera Test
jb = jarque_bera(model2.resid)

print("Statistic :", jb[0])
print("p-value   :", jb[1])
print("Skewness  :", jb[2])
print("Kurtosis   :", jb[3])
```

```
Statistic : 1.3240170834423213
p-value   : 0.5158142588762069
Skewness  : 0.4923761179116778
Kurtosis  : 3.502426760360276
```

In [96]:

```
### Cell 14: Durbin-Watson
dw = durbin_watson(model2.resid)

print(dw)
```

```
0.28162582227632466
```

In [97]:

```
### Cell 15: Breusch-Godfrey
bg = acorr_breusch_godfrey(model2, nlags=1)

print("LM Statistic :", bg[0])
print("LM p-value   :", bg[1])
print("F Statistic  :", bg[2])
print("F p-value    :", bg[3])
```

```
LM Statistic : 19.30767280237088
LM p-value   : 1.1125836030967709e-05
F Statistic  : 63.471015254998555
F p-value    : 6.32384023509354e-08
```

In [99]:

```
# MODEL 3
## Log-Log Demand Model
### Cell 16: Take Logarithms
df2 = np.log(df2[["Quantity",
                  "Price",
                  "Income",
                  "SubPrice",
                  "CompPrice"]])

df2.head()
```

Out[99]:

	Quantity	Price	Income	SubPrice	CompPrice
0	14.500867	2.438071	10.933095	2.608664	2.139128
1	14.352339	2.859893	10.595870	3.196032	1.120377
2	14.067237	2.557826	10.806020	2.945517	2.134576
3	14.071408	2.922958	11.255041	2.875001	2.092572
4	13.502808	2.959359	10.799048	2.774998	2.002955

In [100]:

```
### Cell 17: Prepare the Variables
X = df2[["Price",
         "Income",
         "SubPrice",
         "CompPrice"]]

X = sm.add_constant(X)

y = df2["Quantity"]
```

In [101]:

```
### Cell 18: Fit the Model
model3 = sm.OLS(y, X).fit()

print(model3.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          Quantity    R-squared:                0.910
Model:                  OLS        Adj. R-squared:            0.907
```

Method:	Least Squares	F-statistic:	241.3
Date:	Wed, 01 Jul 2026	Prob (F-statistic):	7.55e-49
Time:	21:54:47	Log-Likelihood:	56.779
No. Observations:	100	AIC:	-103.6
Df Residuals:	95	BIC:	-90.53
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	8.1941	0.547	14.992	0.000	7.109	9.279
Price	-1.1390	0.056	-20.265	0.000	-1.251	-1.027
Income	0.7531	0.043	17.428	0.000	0.667	0.839
SubPrice	0.5367	0.054	9.986	0.000	0.430	0.643
CompPrice	-0.4068	0.042	-9.635	0.000	-0.491	-0.323

Omnibus:	1.702	Durbin-Watson:	1.748
Prob(Omnibus):	0.427	Jarque-Bera (JB):	1.744
Skew:	0.295	Prob(JB):	0.418
Kurtosis:	2.733	Cond. No.	454.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [102]:

```
### Cell 19: VIF
vif = pd.DataFrame()

vif["Variable"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i)
               for i in range(X.shape[1])]

print(vif)
```

	Variable	VIF
0	const	1508.830963
1	Price	1.022532
2	Income	1.019721
3	SubPrice	1.011602
4	CompPrice	1.009046

In [103]:

```
### Cell 20: Breusch-Pagan Test
bp = het_breuschpagan(model3.resid, model3.model.exog)

print("LM Statistic :", bp[0])
print("LM p-value    :", bp[1])
print("F Statistic   :", bp[2])
print("F p-value     :", bp[3])
```

```
LM Statistic : 4.940798291979432
LM p-value    : 0.29342607922282277
F Statistic   : 1.2344303058102657
F p-value     : 0.3015523872070202
```

In [104]:

```
### Cell 21: Jarque-Bera Test
jb = jarque_bera(model3.resid)
```

```
print("Statistic :", jb[0])
print("p-value   :", jb[1])
print("Skewness  :", jb[2])
print("Kurtosis  :", jb[3])
```

```
Statistic : 1.7442323969530866
p-value   : 0.4180659020505163
Skewness  : 0.2947348738024171
Kurtosis  : 2.7332769377660906
```

Notes for Student!!

- Each cell performs **one task only**, making it easier to explain.
- There are **no custom functions**, you can see every computation explicitly.
- Variables such as `X`, `y`, `model1`, `model2`, and `model3` are familiar and easy to follow.
- Each diagnostic test is run independently, allowing you to explain the purpose and interpretation before moving to the next step.
- you can execute the notebook cell by cell and observe the output after every step, which reinforces learning.

In []: